PeopleOps Data Hackathon: HR Pipeline Case Study

Oct 23, 2026 · @Qasim Hassan

The scenario

Your team has two days to turn six months of messy HR exports into a trusted data warehouse, then tell the CFO why overtime cost keeps rising.

The client. Indus Weave Textiles (fictional) is a textile manufacturer with about 1,200 employees across a head office and three plants. It recently moved HR onto a cloud HCM platform modeled on Polarapt HCM. Attendance, shifts, leave, overtime, payroll and recruitment each live in a separate module, and each exports its own files.

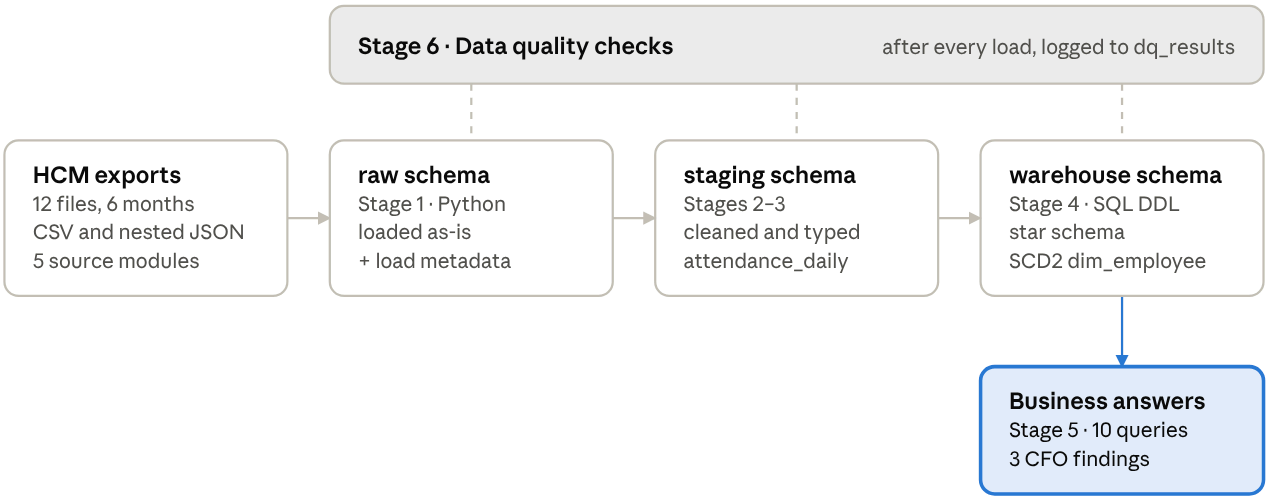
The problem. The overtime bill has climbed for six months. HR says attendance looks normal. Payroll pays overtime from approved OT requests, not from actual biometric punches. Nobody trusts anyone's numbers, because every module tells a slightly different story.

Your mission. Build a Python + SQL pipeline that:

1. Loads raw exports from the HCM modules into a database, untouched.
2. Cleans and reconciles them into one trusted model of employees, attendance, leave and payroll.
3. Answers 10 business questions with SQL.
4. Proves the numbers with data quality checks.
5. Presents the top three findings to the CFO in a 5-minute demo.

Stack. Python 3 (pandas, SQLAlchemy or psycopg2) and PostgreSQL. SQLite is allowed if a team can't run PostgreSQL. Cloud, Spark and BI tools are not needed: this round is about getting the data right.

Raw exports pass through three schemas to answers the CFO can trust



hackathon pipeline · 4 layers, 6 stages

Stages 1 to 5 move data left to right; Stage 6 checks every schema after each load.

Dataset pack

Each team gets the same 12 data files plus a data dictionary, exported from five HCM modules and covering 1 April to 30 September 2026. Mentors share it as `indus_weave_hcm_data.zip` (4.3 MB zipped, 25 MB unzipped). All data is synthetic; no real employee is in it.

File	Format	Source module	Rows	What's messy
`employees.csv`	CSV	Core HR: People	1,316	50 duplicate records, three date formats, department names in mixed case, with stray spaces and old names

File	Format	Source module	Rows	What's messy
`employment_history.csv`	CSV	Core HR: Employment	3,686	Effective-dated hires, promotions, transfers and salary revisions; `effective_to` overlaps or leaves gaps for 37 employees
`departments.csv`	CSV	Foundation: Organization	25	Plant and department hierarchy; two departments were renamed mid-period
`shifts.json`	JSON	Workforce: Shifts	4	Day, evening, night and general shifts; the night shift crosses midnight
`shift_roster.csv`	CSV	Workforce: Rosters	36,402	Weekly shift per employee, April to October; some weeks missing
`device_mapping.csv`	CSV	Workforce: Biometric Gateway	1,260	Device user ID to employee ID
`biometric_punches.csv`	CSV	Workforce: Biometric Gateway	345,280	Raw punches: duplicates seconds apart, missing check-outs, unmapped contractor IDs, one device clock running an hour off
`holidays.csv`	CSV	Workforce: Holidays	12	Public holidays plus plant-specific shutdowns

File	Format	Source module	Rows	What's messy
`leave_requests.json`	Nested JSON	Workforce: Leave	5,372	Approval steps nested inside each request; rejected, pending and cancelled requests; leave spanning Sundays and holidays
`overtime_requests.csv`	CSV	Workforce: Overtime	22,098	Approved, rejected and pending OT per shift date; some approvals for days with no overtime worked
`payroll_lines.csv`	CSV	Payroll: Payslips	53,617	One row per employee, month and pay component (long format); OT paid from approved requests
`applications.csv`	CSV	Talent Acquisition	3,835	Candidate stage changes; used only for the bonus task

Business rules (the full set of 21 rules, with column definitions, is in `data_dictionary.md`):

- If an `emp_id` appears twice in `employees.csv`, keep the row with the latest `updated_at`.
- `effective_to` in the history is unreliable: a record is valid until the day before the employee's next record starts.
- Monthly figures belong to the department the employee was in on the last day of the month (or their last working day).
- A missing roster week means the employee worked the same shift as the previous week.
- Two punches by the same employee within 2 minutes count as one.

- A night-shift punch belongs to the date the shift started, not the calendar date of the punch.
- Late arrival = check-in more than 10 minutes after shift start; early departure = check-out more than 10 minutes before shift end. Count both only on Present working days.
- Overtime = minutes worked after shift end, rounded down to whole half-hours. Work on a Sunday or holiday is all overtime.
- Leave counts only if both approval steps (Manager, then HR) say Approved and it was never cancelled.
- Attendance belongs to Workforce; Payroll consumes attendance results. Your model should reflect that.

Challenge tasks

The work runs in six stages, each building on the last, plus optional bonus tasks worth up to 10 extra points.

Stage 1: Ingest the raw data (Python)

- Load every file into a `raw` schema exactly as delivered, all columns as text. Flatten nothing yet.
- Add load metadata to each table: `_source_file`, `_loaded_at`, `_row_hash`.
- Make the load idempotent: running it twice must not create duplicate rows.
- Log the row count per file and per table, and check they match.

Stage 2: Clean and standardize (Python and SQL)

Build a `staging` schema with typed, cleaned tables.

- Trim text, fix casing, and convert all dates to ISO `YYYY-MM-DD`.
- Deduplicate employees and keep a log of which records were merged and why.
- Map device user IDs to employees. Park unmapped punches in a `staging.rejected_punches` table with a reason.
- Detect and correct the device whose clock is off. Explain how you found it.
- Remove duplicate punches using the 2-minute rule.
- Flatten `leave_requests.json` into one row per request and one row per approval step.
- Expand each approved leave request into one row per leave day.

Stage 3: Apply the business logic (SQL)

Build `staging.attendance_daily`: one row per employee per scheduled day.

- Pair first check-in and last check-out per shift, with night shifts assigned to their start date.
- Calculate worked hours, late minutes, early-departure minutes and overtime hours using the business rules.
- Give every row one status: `Present`, `Absent`, `On Leave`, `Holiday`, `Weekly Off` or `Missing Punch`.

Stage 4: Model the warehouse (SQL DDL)

Design a star schema in a `warehouse` schema, with primary and foreign keys enforced.

- Dimensions: `dim_employee` (SCD Type 2 from employment history), `dim_department`, `dim_shift`, `dim_date`.
- Facts: `fact_attendance_daily`, `fact_leave_day`, `fact_overtime_approved`, `fact_payroll_line`.
- Include an ER diagram (hand-drawn photo, dbdiagram.io or draw.io all fine).

Stage 5: Answer the business questions (SQL)

One `.sql` file per question, using CTEs and window functions where they fit. Save each result as a CSV.

1. Month-end headcount by plant and department, for all six months.
2. Monthly attrition rate by department (separations ÷ average headcount).
3. Absenteeism rate by plant and month, excluding approved leave.
4. The 10 employees with the most late arrivals, and their department.
5. Overtime hours computed from punches vs approved vs paid, by department and month.
6. Departments where paid overtime exceeds computed overtime by more than 15%.
7. Employees paid in a month with zero attendance in that month.
8. Employees still receiving pay after their separation date.
9. Payroll cost per employee by department, with month-over-month change.
10. Employees whose approved Annual leave days in these six months already exceed their full-year entitlement.

Stage 6: Prove it with data quality checks (Python or SQL)

Write automated checks that run after every load and write results to a `dq_results` table.

- Uniqueness and not-null checks on every primary key.
- Referential integrity: every fact row has a matching dimension row.
- Reconciliation: raw row count = staging rows + rejected rows, for every table.
- Accepted values: attendance status, leave type, pay component.
- Business sanity: no negative worked hours, no shift longer than 16 hours, no overlapping SCD2 ranges.

Bonus tasks

- `run_pipeline.py` that rebuilds everything from raw files with one command.
- Incremental load: a seventh month of punches arrives mid-hackathon; load only the new data.
- Recruitment funnel from `applications.csv`: conversion and median days at each stage.
- A simple dashboard in any tool, built on the warehouse tables.

Deliverables

Each team submits one public GitHub repo link before code freeze. Commits after code freeze are not judged.

```
indus-weave-hr-pipeline/
├── README.md            # setup, how to run, design decisions, team members
├── requirements.txt
├── run_pipeline.py      # bonus: one-command rebuild
├── ingestion/           # Stage 1 Python scripts
├── transform/           # Stage 2 and 3 Python + SQL
├── warehouse/
│   ├── ddl/             # Stage 4 CREATE TABLE scripts
│   └── erd.png
├── analytics/
│   ├── q01_headcount.sql … q10_leave_overuse.sql
│   └── results/         # one CSV per question
├── quality/             # Stage 6 checks
└── presentation/
    ├── cfo_findings.md  # top 3 findings, one paragraph each
    └── slides.pdf       # max 6 slides
```

- **README** a mentor can follow to rebuild your database from the raw files on a clean machine.
- **CFO findings**: three findings, each with the number, the evidence query and a recommended action.
- **Demo**: 5 minutes plus 3 minutes of Q&A. Any team member may be asked to explain any line of code.

Rules and timeline

- Teams of 3 or 4. Mentors form teams so each has a mix of strong and newer coders.
- Python and SQL only for Stages 1 to 6. Any library on PyPI is allowed.
- AI assistants are allowed, but the team must be able to explain every line in Q&A. Code nobody can explain scores zero for that stage.
- Do not edit the raw files by hand. Every fix must be in code.
- Ask mentors about business rules, never about solutions.

When	Milestone
Day 1, morning	Kickoff, dataset walkthrough, teams formed
Day 1, end of day	Checkpoint 1: raw and staging loaded, row counts shown to a mentor

When	Milestone
Day 2, midday	Checkpoint 2: warehouse built, at least 5 questions answered
Day 2, mid-afternoon	Code freeze, repo link submitted
Day 2, late afternoon	Demos, judging, results

Judging rubric

Correct numbers carry the most weight; mentors check every answer against a hidden answer key built from the same data.

Criterion	Points	Full marks look like
Correct answers (Stage 5)	25	All 10 results match the answer key within rounding
Cleaning and business logic (Stages 2–3)	20	Every planted data issue found, fixed in code and documented
Data model (Stage 4)	15	Clean star schema, correct SCD2, keys enforced, clear ERD
Data quality and reconciliation (Stage 6)	15	Checks run automatically and every raw row is accounted for
CFO findings and demo	15	Three findings with numbers, evidence and an action; clear 5-minute story
Code quality and reproducibility (Stage 1, README)	10	Idempotent load, readable code, a mentor rebuilds it from the README
Total	**100**	Bonus tasks add up to 10 more

Hints

- Profile before you clean. Run `df.describe()`, `value_counts()` and null counts on every file first.

- Removing duplicate punches is a `LAG()` problem: compare each punch with the previous one for the same employee.
- For night shifts, shift each punch back by a few hours before taking its date, then pair check-in and check-out.
- To find the faulty device, compare average check-in time per device against the shift start.
- SCD Type 2 joins look like `fact.work_date BETWEEN dim.valid_from AND dim.valid_to`.
- Build `dim_date` with `generate_series()` in PostgreSQL, or a pandas date range for SQLite.
- `payroll_lines.csv` is long format. Pivot it with `SUM(CASE WHEN component = 'OT' THEN amount END)`.
- Questions 5 to 8 are where the CFO's story is hiding. Start your findings there.